

Module 5 — Gestion des requêtes HTTP avec Cypress

Objectif du module

À la fin de ce module, l'étudiant saura :

- ce qu'est une **requête HTTP**
 - la différence entre :
 - tester une API
 - écouter une requête réseau
 - stubber (mock) une requête
 - utiliser :
 - `cy.request()`
 - `cy.intercept()`
 - `cy.wait()`
 - comprendre **à quoi ça sert en entreprise**
 - écrire des tests **stables, réalistes et professionnels**
-

Rappel fondamental (à dire à l'étudiant)

Une application web fonctionne grâce à des **échanges réseau** (le navigateur parle à un serveur).

Chaque clic, chaque chargement de page peut déclencher une **requête HTTP**.

Où créer les fichiers (important)

Tous les fichiers de ce module doivent être créés ici :

```
cypress/e2e/module5-requetes-http/  
├── exemples/  
│   ├── api_request.cy.js  
│   ├── intercept_observer.cy.js  
│   └── intercept_stub.cy.js  
└── exercices/  
    ├── exercice_request.cy.js  
    └── exercice_intercept.cy.js
```

Règle d'or

 Un fichier = une idée

👉 Un exemple = un fichier

👉 Un exercice = un fichier

1 Tester une API directement avec `cy.request()`

🧠 C'est quoi `cy.request()` ?

Phrase à retenir

`cy.request()` permet de parler directement à une API, sans passer par l'interface.

➡ Pas de navigateur

➡ Pas de clic

➡ Juste une requête HTTP

Exemple 1 — Tester une API publique

📄 Fichier :

```
cypress/e2e/module5-requetes-http/exemples/api_request.cy.js
describe("Tester une API avec cy.request", () => {
  it("doit retourner une liste de posts", () => {
    cy.request("https://jsonplaceholder.typicode.com/posts")
      .should((response) => {
        expect(response.status).to.eq(200);
        expect(response.body).to.have.length.greaterThan(0);
      });
  });
});
```

🔍 Explication ligne par ligne

- `cy.request(url)` → envoie une requête HTTP GET
- `response.status` → code HTTP (200 = OK)
- `response.body` → données renvoyées par l'API

👉 Ici on teste le backend uniquement



Cas réel entreprise

- Vérifier qu'une API répond
- Vérifier un contrat de données
- Tester une API indépendamment du front

2 Écouter une requête réseau avec `cy.intercept()` (sans stub)



Qu'est-ce qu'“écouter une requête” ?

Phrase simple

Écouter une requête = regarder passer une requête réelle déclenchée par l'UI

👉 Cypress ne modifie rien

👉 Il observe seulement

Exemple 2 — Observer une requête réseau réelle



Fichier :

```
cypress/e2e/module5-requetes-http/exemples/intercept_observer.cy.js
describe("Observer une requête réseau", () => {
  it("doit écouter l'appel API déclenché par la page", () => {
    // On prépare l'écoute sans fournir de corps de réponse (pas de stub)
    cy.intercept(
      "GET",
      "**/comments/1" // On observe l'appel vers l'API des commentaires
    [cite: 68]
    ).as("getComment"); // On définit un alias pour le suivi

    // Navigation sur la page
    cy.visit("https://example.cypress.io/commands/network-requests");

    // L'action utilisateur déclenche une requête réelle
    cy.contains("Get Comment").click();

    // On attend que la requête réelle revienne du serveur
    cy.wait("@getComment")
      .its("response.statusCode") // On extrait le code de statut de la
    réponse
```

```
        .should("eq", 200); // On vérifie que le serveur a bien répondu 200
    OK
    });
  });
```

Ce qui se passe réellement

1. L'utilisateur clique
 2. Le navigateur fait une requête API
 3. Cypress **écoute**
 4. Cypress vérifie la réponse
-

Pourquoi `cy.wait()` ?

Phrase clé

On attend que la requête réseau se termine avant de vérifier quoi que ce soit.

- Cypress attend la réponse
 - Timeout par défaut **≈ 5 secondes**
 - Sinon le test échoue
-

Cas réel entreprise

- Vérifier qu'un clic déclenche bien une API
 - Vérifier un code HTTP
 - Vérifier que l'UI dépend bien de la réponse
-

3 Stubber (mock) une requête avec `cy.intercept()`

Phrase la plus importante du module

Stubber une requête, c'est empêcher l'appel réel et fournir une fausse réponse contrôlée.

Exemple 3 — Stub d'une requête API

Fichier :

```
cypress/e2e/module5-requetes-http/exemples/intercept_stub.cy.js
describe("Stubber une requête API", () => {
  it("doit afficher des données simulées", () => {
    cy.intercept(
      "GET",
      "https://jsonplaceholder.typicode.com/posts",
      {
        statusCode: 200,
        body: [
          { id: 1, title: "Post simulé", body: "Contenu fake" }
        ]
      }
    ).as("fakePosts");

    cy.visit("https://example.cypress.io/commands/network-requests");

    cy.contains("Get Comment").click();

    cy.wait("@fakePosts");

    cy.contains("Post simulé").should("exist");
  });
});
```

Ce que Cypress fait ici

- L'API réelle n'est **jamais** appelée
 - Cypress répond à la place du serveur
 - L'UI croit que tout est normal
-

Cas d'usage entreprise (ultra réalistes)

- Backend indisponible
 - Données instables
 - Tester des erreurs (500, 404)
 - Tester des cas rares
 - Tests rapides et fiables
-

Tester la création et la modification (POST & PATCH)

Rappel des méthodes

- **POST** : Sert à **créer** une ressource (ex: envoyer un formulaire).
- **PATCH** : Sert à **modifier** partiellement une ressource existante.

Exemple 4 — Tester un POST (Création) avec cy.request()



Fichier :

```
cypress/e2e/module5-requetes-http/exemples/api_post.cy.js
```

```
describe("Tester une création avec POST", () => {
  it("doit créer un nouveau post sur l'API", () => {
    cy.request("POST", "https://jsonplaceholder.typicode.com/posts", {
      title: "Cypress Avancé",
      body: "Apprendre les requêtes HTTP",
      userId: 1
    }).should((response) => {
      expect(response.status).to.eq(201); // 201 = Création réussie
      expect(response.body.title).to.eq("Cypress Avancé");
    });
  });
});
```

Exemple 5 — Stubber un POST avec cy.intercept()



Fichier :

```
cypress/e2e/module5-requetes-http/exemples/intercept_post_stub.cy.js
```

```
describe("Stubbing de création", () => {
  it("doit simuler la réussite d'un envoi de formulaire", () => {
    cy.intercept("POST", "**/comments", {
      statusCode: 201,
      body: { id: 501, name: "Succès" }
    }).as("postComment");

    cy.visit("https://example.cypress.io/commands/network-requests");
    cy.contains("Post Comment").click();

    cy.wait("@postComment"); // On attend que l'UI envoie les données
    [cite: 82]
    cy.get(".network-post-comment").should("contain", "POST Successful!");
  });
});
```

4 Différence API vs Requête réseau (très important)

Type	Outil	Ce qu'on teste
API seule	<code>cy.request</code>	Backend
Réseau réel	<code>cy.intercept</code> (sans stub)	UI + API
Stub	<code>cy.intercept</code> (avec réponse)	UI uniquement

5 Exercices pratiques

Exercice 1 — Tester une API

 **Fichier :**

`cypress/e2e/module5-requetes-http/exercices/exercice_request.cy.js`

 **Consigne**

1. Appeler l'API posts
2. Vérifier le code 200
3. Vérifier qu'il y a au moins un élément

 **Correction**

Exercice 2 — Stub d'une requête

 **Fichier :**

`cypress/e2e/module5-requetes-http/exercices/exercice_intercept.cy.js`

 **Consigne**

1. Stubber une requête GET

2. Retourner une réponse fake
3. Vérifier que l'UI affiche la donnée simulée

✓ Correction

Exercice 3 — Modifier une donnée (PATCH)

 **Fichier :**

cypress/e2e/module5-requetes-http/exercices/exercice_patch.cy.js

 **Consigne :**

1. Utiliser `cy.request` pour modifier le titre du post n°1.
2. Vérifier que le code retour est 200.
3. Vérifier que le titre a bien été mis à jour dans la réponse.

✓ **Correction :**

Pièges à éviter

- ✗ Tester le backend avec des stubs
- ✗ Oublier `cy.wait()`
- ✗ Mélanger données et logique
- ✗ Tout stubber sans raison

Règle d'or entreprise

On stub pour tester l'UI.

On ne stub pas pour tester le backend.

✓ Conclusion du module

À ce stade, l'étudiant sait :

- tester une API
- comprendre les appels réseau

- contrôler les réponses
- écrire des tests professionnels
- comprendre ce qu'il fera en mission